Software Construction & Development (SCD) — Lab Tasks

Design Patterns Implementation & Semester Project Architecture Blueprint

STUDENT NAME	ROLL NUMBER	SUBJECT	PROJECT SCOPE
Abdul Munib	FA24-BSSE-002	SCD Laboratory	Online Examination System

TASK 1 (PART A): SINGLETON & FACTORY DESIGN PATTERNS (JAVA)

All source files are organized into standard modular packages for direct import into an IDE project (NetBeans/Eclipse). Each pattern addresses object creation decoupling and centralized instance control.

singleton/SingleObject.java

```
package singleton; public class SingleOb
ject { private static SingleObject instance = ne
w SingleObject(); private SingleObject() {} publ
ic static SingleObject getInstance() { return in
stance; } public void showMessage() { System.ou
t.println("Hello World!"); } }
```

singleton/SingletonPatternDemo.java

```
package singleton; public class Singleto
nPatternDemo { public static void main(String[]
args) { // SingleObject obj = new SingleObject
(); // Compile Error SingleObject object = Singl
eObject.getInstance(); object.showMessage(); } }
```

factory/Shape.java & Concrete Classes

```
package factory; public interface Shape
{ void draw(); } class Rectangle implements Shap
e { public void draw() { System.out.println("Ins
ide Rectangle::draw()"); } } class Square implem
ents Shape { public void draw() { System.out.pri
ntln("Inside Square::draw()"); } } class Circle
implements Shape { public void draw() { System.o
ut.println("Inside Circle::draw()"); } }
```

factory/ShapeFactory.java & Demo

```
package factory; public class ShapeFacto
ry { public Shape getShape(String type) { if (ty
pe == null) return null; if (type.equalsIgnoreCa
se("CIRCLE")) return new Circle(); if (type.equa
lsIgnoreCase("RECTANGLE")) return new Rectangle
(); if (type.equalsIgnoreCase("SQUARE")) return
new Square(); return null; } } // Demo execution
calls getShape() with polymorphic draw()
```

TASK 1 (PART B): FACADE & MVC DESIGN PATTERNS (JAVA)

Structural and architectural decoupling using the Facade pattern (hiding complexity behind a unified API) and the Model-View-Controller pattern (isolating business entities from presentation mechanics).

facade/ShapeMaker.java (Facade Interface)

```
package facade; public class ShapeMaker { private Shape circle; private Shape rectangle; private Shape square; public ShapeMaker() { circle = new Circle(); rectangle = new Rectangle(); square = new Square(); } public void drawCircle() { circle.draw(); } public void drawRectangle() { rectangle.draw(); } public void drawSquare() { square.draw(); } } public class FacadePatternDemo { public static void main(String[] args) { ShapeMaker shapeMaker = new ShapeMaker(); shapeMaker.drawCircle(); shapeMaker.drawRectangle(); shapeMaker.drawSquare(); } }
```

mvc/Student.java & StudentView.java

```
package mvc; public class Student { private String rollNo, name; public String getRollNo() { return rollNo; } public void setRollNo(String r) { this.rollNo = r; } public String getName() { return name; } public void setName(String n) { this.name = n; } } public class StudentView { public void printStudentDetails(String name, String rollNo) { System.out.println("Student: "); System.out.println("Name: " + name); System.out.println("Roll No: " + rollNo); } }
```

mvc/StudentController.java & MVCPatternDemo.java

```
package mvc; public class StudentController { private Student model; private StudentView view; public StudentController(Student m, StudentView v) { this.model = m; this.view = v; } public void setStudentName(String name) { model.setName(name); } public String getStudentName() { return model.getName(); } public void setStudentRollNo(String roll) { model.setRollNo(roll); } public String getStudentRollNo() { return model.getRollNo(); } public void updateView() { view.printStudentDetails(model.getName(), model.getRollNo()); } } public class MVCPatternDemo { public static void main(String[] args) { Student model = new Student(); model.setName("Robert"); model.setRollNo("10"); StudentView view = new StudentView(); StudentController controller = new StudentController(model, view); controller.updateView(); controller.setStudentName("John"); controller.updateView(); } }
```

TASK 2: PATTERN IDENTIFICATION FOR SEMESTER PROJECT (10 MARKS)

System Name: Online Examination, Assessment & Evaluation System (Abdul Munib — FA24-BSSE-002). Below is the precise mapping of Gang of Four (GoF) design patterns to core modules defined in the project proposal.

Creational 1. Singleton Pattern — DatabaseConnectionManager & RedisSessionPool

Application: Manages persistent connections to PostgreSQL and active Redis state caching.
Justification: Concurrent exam candidates fire state-recovery requests every 30 seconds. A single centralized connection pool prevents resource starvation, ensures thread safety, and avoids database exhaustion.

Creational 2. Factory Method Pattern — QuestionFactory Module

Application: Instantiates dynamic question entities: `MCQQuestion`, `CodingQuestion`, and `SubjectiveQuestion`.
Justification: The assessment builder ingests diverse question banks with differing cognitive tags (Bloom's Taxonomy) and submission criteria. Decouples the exam generator from concrete question subclasses.

Behavioral 3. Strategy Pattern — Evaluation & Grading Engine

Application: Encapsulates scoring algorithms: `AutoObjectiveStrategy`, `RubricSubjectiveStrategy`, and `UnitTestCaseStrategy`.
Justification: Enables dynamic switching of grading schemes at runtime based on assessment configurations without modifying core evaluation orchestrators (Open/Closed Principle).

Structural 4. Facade Pattern — ExamSubmissionFacade

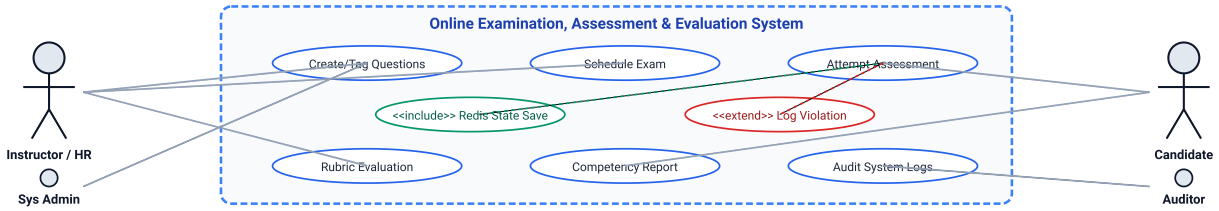
Application: Consolidates multi-step exam termination routines into a singular `submitExam(sessionId)` call.
Justification: Submissions trigger timer termination, proctor log analysis, auto-evaluation dispatch, Redis session flushing, and audit trail commitment. The facade shields API controllers from complex internal orchestrations.

Behavioral 5. Observer Pattern — Proctoring Violation Event Dispatcher

Application: Triggers alerts across UI, supervisor feeds, and logging storage when anti-cheat events fire.
Justification: Client heuristics (tab-switch, full-screen breach, clipboard copy) act as subjects; the logger, live monitor, and penalty counter act as detached observers responding asynchronously.

TASK 3 (PART A): USE CASE DIAGRAM & ACTOR SPECIFICATIONS

The Use Case diagram visualizes boundary interactions between external actors and system capabilities across academic and corporate environments.

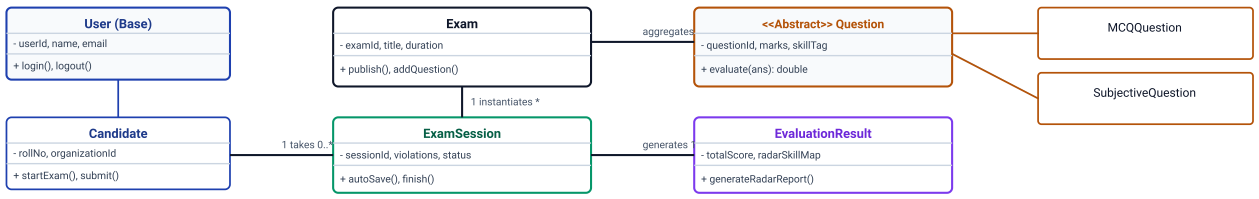


Use Case Specification Matrix

Use Case Name	Primary Actor	Type	Description & Business Rules
Manage Question Bank	Instructor / HR	Primary	Allows dynamic authoring, bulk CSV import, and tagging by Bloom's Taxonomy, skill dimension, and difficulty.
Conduct Assessment	Candidate	Primary	Delivers active exam UI with auto-advance, responsive inputs, and real-time timer synchronization.
State Persistence	Exam Engine	<<include>>	Caches candidate response states into Redis cache every 30 seconds to support seamless connection recovery.
Heuristic Proctoring	Proctoring Service	<<extend>>	Monitors full-screen compliance, clipboard lock, and tab transitions, triggering penalty strikes on breach.
Competency Analytics	Instructor / Student	Primary	Computes granular domain metrics and generates multi-dimensional radar skill charts beyond simple raw marks.

TASK 3 (PART B): COMPLETE DOMAIN CLASS DIAGRAM

Structural object model capturing inheritance, associations, and operational signatures across the assessment execution lifecycle.



Detailed Class Dictionary & Architectural Roles

Class Name	Stereotype / Layer	Key Attributes	Responsibilities & Collaborators
`User`	Abstract Base Entity	`userId, name, email, role`	Handles credentials and role-based permissions (Candidate, Instructor, Admin).
`Exam`	Domain Aggregate Root	`examId, duration, passMarks`	Contains references to aggregated `Question` entities; enforces exam settings.
`Question`	Abstract Class (Factory Target)	`questionId, maxMarks, cognitiveTag`	Defines contract for polymorphic scoring algorithms and taxonomy indexing.
`ExamSession`	Stateful Runtime Entity	`sessionId, violations, redisKey`	Bridges the active candidate exam attempt to Redis caching for fault tolerance.
`EvaluationResult`	Analytics Aggregate	`percentage, skillMatrixMap`	Computes competency matrices across taxonomy tags to produce analytical radar outputs.